

Project Overview

U1L6 Wasteland Adventure Project – Overview & Level Guide

Choose your path. Build your world. Survive your story.

Welcome to the Wasteland Adventure!

In this mini-project, you're stepping into a post-apocalyptic world where **your choices, your creativity,** and **your Python skills** shape the journey.

You'll build:

- Your **survival backpack** using lists
- A **map of the wasteland** using strings or a 2D grid
- A **story or expedition log** using f-strings
- And maybe even an interactive loop or advanced features if you level up...

You choose the difficulty level.

You choose how deep it goes.

You choose how epic your world becomes.

This is your story. Let's make it unforgettable.

You can use the attached resources as well as any of your previous notes and materials to help you.

Resources for Any Level:

- [Story Help](#)
- [Rubric](#)
- [Level Up Any Time](#)– take on more of a challenge at any point in the project!

Choose Your Level: Starter, Power-Up, or Legendary

1. Starter Mode

Perfect if you're still building confidence with:

- Lists
- Indexing
- `append()`, `len()`
- Simple printing & formatting

You'll build:

- A **simple backpack list**
- A **basic 3×3 or 5×5 map**
- A short **story snippet** describing your adventure

Starter Mode is clean, focused, and gets the job done.

No loops. No input needed. No stress.

Just you, Python, and a world you're creating one `print()` at a time.

This mode is great if you want to master the fundamentals and walk away proud of solid, clean code.

2. Power-Up Mode

For students who feel ready to push themselves a little further.

You'll add:

- A bigger **5×5 or 10×10 map**
- Basic **`input()`** to collect player items
- Some formatting tricks with strings or f-strings
- Optional extras like small icons, characters, or events

Power-Up Mode lets you stretch beyond the basics **without going full Legendary**.

It's a sweet spot between comfort and challenge.

Choose this if you're ready to level up and explore more advanced tools while still staying in your comfort zone.

3. Legendary Mode

You want challenge? You want creativity?

This is the full adventure.

You'll build:

- A **10×10 wasteland map**
- A dynamic **backpack system with a while loop**
- Optional: remove(), searching, sorting
- Named locations with **full lore**
- A final story tying everything together
- Advanced formatting tricks or even a 2D list grid

You will write all your own code — no skeleton provided except comments.

Your creativity sets the ceiling.

Your logic will push the limits of what you've learned.

This mode is where worldbuilders, storytellers, and coders shine.

If you want to feel like you're coding a mini-game, come here.

Leveling Up Anytime

You are **never locked** into a difficulty.

If Starter feels too easy → jump to Power-Up.

If Power-Up feels exciting → switch to Legendary.

If Legendary is too intense → your previous level is always available.

Your project will still be graded fairly **based on the level you choose**.

Do what helps you learn the most.

What Will You Create?

Will your backpack save your life?

Will your map lead to hidden safe zones?

Will your story unveil secrets buried beneath the sands of the wasteland?

Or will you craft something totally unexpected?

Whatever you choose...

Make it creative.

Make it yours.

And have fun — you're building a world from scratch.

Story Help

Story Support Guide

Here are helpful ideas to make your wasteland story fun, creative, and personal. You only need a few sentences, but you can go bigger if inspiration hits.

Build Your World

You can describe things like:

- What the wasteland looks, sounds, or feels like
- Strange weather, creatures, shadows, or abandoned places
- Ruins, old signs, crashed vehicles, or mysterious footprints
- What the safe zone marked "X" might be like

Examples:

- "The air shimmered with heat, and each step kicked up pale dust."
- "I kept hearing rustling, but I couldn't tell if it was wind or something watching me."

Talk About Your Backpack

You can:

- Explain why you packed certain items
- Describe how an item helps you
- Mention one using indexing like `backpack[2]`
- Highlight something surprising or important

Examples:

- "Thankfully, my `{backpack[2]}` might be useful if things go wrong."
- "I regretted removing the water purifier... but it was too heavy."

Use Your Map in the Story

Try describing at least one location:

- What is "1" on your map?
- What makes the safe zone interesting or dangerous?
- Why is the map hand-drawn?
- Who gave you the map?

Examples:

- "Location 1 looked harmless on the map, but everyone knew the story of the old radio tower."
- "The safe zone was rumored to have actual green plants — the first in years."

Choose Any Tone You Want

Your story can be:

- dramatic
- funny
- mysterious
- hopeful
- chaotic
- adventurous

Examples:

- Funny: "My boots were too big, the sun was too bright, and I was 100% lost."
- Dramatic: "If I didn't reach the safe zone by sundown, I wasn't sure I'd make it through the night."

Add a Small Twist (Optional)

These can make your story stand out:

- Something is following you
- Part of the map is wrong
- You uncover a secret at location 1
- You lose or break an item
- You meet a stranger
- The safe zone isn't what you expected

Use your imagination — this is your world.

Rubric

U1L6 Project Rubric (40 Points Total)

Category	Excellent (Full Points)	Strong	Developing	Beginning	No Evidence	Points
A. Code Functionality & No Errors (15 pts)	15 pts Code runs perfectly with no errors. All features work as intended.	12 pts Minor issues, still runs. Most features work.	9 pts Some noticeable issues, but code mostly runs.	6 pts Major issues; only parts run correctly.	0-3 pts Does not run or missing work.	/15
B. Backpack Requirements (8 pts)	8 pts All backpack tasks complete (list, indexing, append, remove, len, formatting).	6 pts Most tasks complete; one part missing.	4 pts Several parts missing or incorrect.	2 pts Minimal attempt.	0 pts No backpack code.	/8
C. Map Creation (8 pts)	8 pts Map meets all requirements for chosen level (3×3 / 5×5 / 10×10). Clear formatting.	6 pts Map mostly clear; minor spacing/formatting issues.	4 pts Missing symbols or messy formatting.	2 pts Attempted but not functional.	0 pts No map.	/8
D. Story / Description (6 pts)	6 pts Meets all story requirements for level. Creative, includes indexed item correctly.	4 pts Meets most requirements; may be brief or missing detail.	2 pts Includes some story; missing major requirements.	1 pt Very incomplete.	0 pts No story.	/6
E. Challenge & Effort (3 pts)	3 pts Clear effort, creativity, leveling up, or personal touches.	2 pts Meets expectations.	1 pt Minimal effort.	–	0 pts	/3

Level Up Any Time

Level Up Any Time Guide

You do not have to stay stuck at one difficulty. You can upgrade your project whenever you feel ready, without throwing your work away.

Think of the levels like this:

- Starter: core basics
- Power-Up: more control and creativity
- Legendary: full adventure with loops and a big map

Starter → Power-Up

You are ready to move from Starter to Power-Up if:

- You can create a list and use indexing
- You can use `append()`, `remove()`, and `len()`
- Your 3×3 map prints correctly
- You wrote a short story

To level up:

- Keep your backpack list, but add `input()` so the user can choose some items
- Expand your map from 3×3 to 5×5
- Add more locations like 2 and 3
- Add lore with `input()` (ask the user to name places and describe them)

You do not need to restart your file. You just add new sections and improve what you already have.

Power-Up → Legendary

You are ready to try Legendary if:

- You feel comfortable with lists and indexing
- You can handle `input()` easily
- You want a bigger challenge

To level up:

- Replace your manual “add items” section with a `while` loop that keeps adding items until the user says “no”
- Upgrade your map from 5×5 to 10×10
- Try storing the map in a grid (2D list) if you want extra power
- Add more detailed lore for locations 1, 2, 3, and X
- Expand your story so it feels like the start of a real adventure

Again, you can copy your old backpack ideas and map layout. You are not starting over, just upgrading.

Already Legendary?

If you are already at Legendary and want more:

- Add a simple “path” on the map marked with *
- Add random events
- Track health or stamina as variables
- Add more locations or secrets
- Use loops to simulate travel steps

You can treat new ideas as “expansion packs” to your existing project.

STARTER

U1L6 Project – STARTER MODE

"Your First Steps Into the Wasteland"

Welcome, survivor. You've just received your first backpack and a small hand-drawn map from a traveler passing through your settlement.

Your job is to prepare for a short journey into the wasteland. This version of the project focuses on the basics: **lists, indexing, f-strings, append, remove, len, and simple map creation**.

This mode is designed so **everyone can succeed**, even if you're still getting comfortable with Python. You'll create a backpack, build a small map, and write a short story — all while practicing your core programming skills.

You can use the attached resources as well as any of your previous notes and materials to help you.

PART 1 – Build Your Starter Backpack

You'll start your adventure by packing the essentials.

Your tasks

1. **Create a list** called `backpack` with **at least 5 items**.
Think: bandages, rope, water bottle, flashlight, journal, etc.
2. Print:
 - The **first item** (`backpack[0]`)
 - The **third item** (`backpack[2]`)
 - The **last item** (`backpack[-1]`, using a negative index)
3. Use `len(backpack)` to print **how many items** you packed.
4. Add **one new item** using `.append()` and print the updated list.
5. Remove **one item** using `.remove()` and print the updated list again.

Helpful Tip

Use f-strings to make your printing look nice.

Example:

```
print(f"You packed {len(backpack)} items.")
```

Starter Skeleton (comments only – you write the code!)

Python

```
# -----  
# BACKPACK SETUP  
# -----  
# TODO: Create a backpack list with at least 5 survival items
```

```
# backpack = [...]
```

```
# TODO: Print the first, third, and last items using indexing
```

```
# TODO: Print how many items are in the backpack using len()
```

```
# TODO: Add an item with append(), then print the list
```

```
# TODO: Remove an item with remove(), then print the new list
```

PART 2 — Create a Simple 3×3 Wasteland Map

The traveler also gives you a **tiny hand-drawn map** that marks:

- 0 — your starting location
- X — the final safe zone
- 1 — a point of interest (your choice!)
- . — empty wasteland

Your tasks

1. Create **three strings**, each representing one row of the 3×3 map.
2. Decide where 0, X, and 1 should be placed.
3. Print the map line by line.
4. Print a **map key** using f-strings, describing each symbol.

Example (but make your own!):

```
. . 1
. 0 .
X . .
```

Starter Skeleton— add to your *current* Python file

```
Python
# -----
# SIMPLE 3x3 MAP
# -----

# TODO: Create three strings representing map rows
# row1 = "..."
# row2 = "..."
# row3 = "..."

# TODO: Print the map, one row per line
```

```
# TODO: Print a map key describing 0, X, 1, and .
```

PART 3 — Write a Short Wasteland Story

You've prepared your gear and studied the map. Now tell the beginning of your journey. If you need help figuring out how to write your story, use the [Story Help](#) tab.

✓ Your story must include:

- the **name of the final safe zone** (X)
- how many items are in your backpack
- at least **one item** mentioned using indexing (`backpack[-1]`)
- a description of location **1**

Your story can be **funny, mysterious, serious, or chaotic** — your world, your tone.

Example starter sentences (optional, these are for inspiration):

- "With each step into the dusty wasteland, I hoped the items in my backpack would be enough to get me through the day."
- "I tightened the straps on my backpack and stared at the strange symbol marked '1' on the map — what could be there?"
- "I looked down at my backpack. It wasn't much, but it was everything I had to survive out here."

Be creative and have fun!

List Basics

List Basics

What is a list?

A list stores multiple things inside one variable.

```
backpack = ["water", "rope", "flashlight", "snacks"]
```

How to print the entire list:

```
print(backpack)
```

How to access one item:

```
print(backpack[0])    # first item  
print(backpack[2])    # third item
```

How to get the *last* item:

```
print(backpack[-1])
```

Indexing

Indexing

How positive indexes work

Index numbers start at **0**, not 1:

Index	Item
0	backpack[0]
1	backpack[1]
2	backpack[2]

Negative indexes

Negative numbers let you reach from the back:

Index	Meaning
-1	last item
-2	second to last
-3	third to last

Example:

```
backpack = ["water", "rope", "flashlight"]  
print(backpack[-1]) # flashlight
```

Map Templates

3×3 Map Templates

You only need 3 rows. Each row is a string.

Template 1 – Fill in your own

```
row1 = ". . ."  
row2 = ". . ."  
row3 = ". . ."
```

Template 2 – With placeholders for 0, X, 1

```
row1 = ". . 1"  
row2 = ". 0 ."  
row3 = "X . ."
```

Template 3 – Open world

Put O, X, and 1 anywhere you want.

```
row1 = ". 0 ."  
row2 = ". . 1"  
row3 = "X . ."
```

How to print the map

```
print(row1)  
print(row2)  
print(row3)
```

Map Key Example

```
0 = Your location  
X = Final safe zone  
1 = Mysterious location  
. = Wasteland
```

f-String Quick Guide

f-String Quick Guide

Basic f-string

```
name = "Jay"  
print(f"Hello, {name}!")
```

Text + list information

```
print(f"You packed {len(backpack)} items.")
```

Text + specific item

```
print(f"The last item in your backpack is {backpack[-1]}.")
```

Print multiple items

```
print(f"You are carrying {backpack[0]} and {backpack[-1]}.")
```

POWER-UP

U1L6 Mini Project – POWER-UP MODE

“Venturing Deeper Into the Wasteland”

You’ve moved beyond basic survival. You’re now preparing for a longer journey through the wasteland, guided by a mysterious traveler’s map. This version introduces **user input**, a **bigger map**, and richer **lore** while still keeping everything friendly for learners who aren’t ready for loops.

This mode builds on the Starter level but gives you more creative control and more tools to shape your survival story.

You’ll gather custom backpack items, design a 5×5 wasteland map, and tell a more detailed story about your journey.

You can use the attached resources as well as any of your previous notes and materials to help you.

PART 1 – The Semi-Custom Backpack

You’ll start with a few essential items, then **ask the user** to choose more.

Your tasks

1. Create a list called `backpack` with **3 items you choose**.
These can be basic survival tools or fun thematic items.
2. Ask the user for **three survival items** using `input()`:
 - 1 special item
 - 1 medical item
 - 1 food/water item
3. Add all three user responses using `.append()`.
4. Print the full backpack using f-strings.
5. Ask the user which item they want to remove
→ Remove it with `.remove(item)` and print the updated backpack.
6. Access items using indexing:
 - `backpack[0]`
 - `backpack[-1]`
 - `backpack[-2]`

Starter code (comments) on the next page

Power-Up Starter Skeleton (comments only– you fill in the code)

Python

```
# -----  
# BACKPACK WITH INPUT  
# -----  
  
# Start with 3 preset items  
# backpack = [...]  
  
# Ask user for:  
# - a special item  
# - a medical item  
# - a food/water item  
# Each answer should be appended to the backpack  
  
# TODO: Print the full backpack using an f-string  
  
# TODO: Ask which item to remove  
# TODO: Use remove() to remove it  
  
# TODO: Print backpack[0], backpack[-1], and backpack[-2]
```

PART 2 – Create a 5×5 Wasteland Map

Your new map will include **multiple numbered locations**, a starting point, and a safe zone.

✓ Your map must include:

- 0 — your starting point
- X — final safe zone
- 1, 2, 3 — three important locations you'll describe

Each row of your map should be created as a **string**, like this:

```
. . 1 . .  
. . . . .  
0 . . . .  
. . 2 . .  
X . . . 3
```

You choose where everything goes.

Additional tasks

1. Create **five strings**, each representing a row of the 5×5 map.
2. Print your map row by row.
3. Ask the user to **name and describe the lore** of locations 1, 2, and 3.
4. Ask the user for the name + lore of the **final safe zone (X)**.
5. Print a map key using f-strings.

Power-Up Map Skeleton (add to your existing code)

Python

```
# -----
# 5x5 MAP WITH LORE
# -----

# TODO: Create five rows as strings (place 0, X, 1, 2, 3)
# row1 = ...
# row2 = ...
# row3 = ...
# row4 = ...
# row5 = ...

# TODO: Print all map rows

# Ask user to name/describe locations 1, 2, and 3
# TODO: loc1_name = input(...)
# TODO: loc1_lore = input(...)
# (repeat for 2 and 3)

# Ask for safe zone information
# TODO: x_name = input("Name your safe zone: ")
# TODO: x_desc = input("Describe it: ")

# TODO: Print a map key using f-strings
```

PART 3 – Power-Up Story

Your story is longer and more detailed than Starter Mode.

✓ Your story must include:

- the name of the safe zone (X)
- at least **two items from your backpack**
 - one must use indexing (`backpack[-1]`)
- a description of **one numbered location** (1, 2, or 3)
- how many total items you're carrying (using `len(backpack)`)
- 4–6 sentences of descriptive writing

You can pick your tone:

- serious

- chaotic
- funny
- mysterious
- dramatic
- hopeful
- sci-fi

You can even include:

- a rumor about one of the locations
- a strange event that happens along the way
- a special purpose for one of your items

Make it yours. Be creative.

OPTIONAL LEVEL-UP IDEAS (Easy additions)

You may add these if you want more challenge:

- Draw a **border** around your map
- Use creative spacing or ASCII art
- Add more numbered locations
- Hide extra symbols on the map
- Invent a legend, a clan, a threat, or a story twist
- Give your character a backstory
- Add weather conditions

Remember: **none of these are required for full credit**, but they can help you earn effort points.

append(), remove(), and len() (power-up_

Mini Guide: `append()`, `remove()`, and `len()`

These three list tools will show up everywhere in your Python projects.

1. `append()`: Add an item to the end

`append()` sticks a new item onto the end of your list.

```
backpack = ["water", "rope", "flashlight"]
```

```
backpack.append("first aid kit")
```

```
print(backpack)
```

```
# ["water", "rope", "flashlight", "first aid kit"]
```

Use `append()` when:

- You want to grow your list one item at a time
- The user gives you a new item with `input()`
- You are collecting options, choices, or inventory

2. `remove()`: Take out a specific item

`remove()` deletes the first match of that value from the list.

```
backpack = ["water", "rope", "flashlight", "rope"]
```

```
backpack.remove("rope")
```

```
print(backpack)
```

```
# ["water", "flashlight", "rope"]
```

Notes:

- It looks for the value, not the index
- It only removes the first match
- If the item is not in the list, you will get an error

Safer version:

```
item_to_remove = "rope"
```

```
if item_to_remove in backpack:
```

```
    backpack.remove(item_to_remove)
```

```
else:
```

```
    print(f"{item_to_remove} is not in your backpack.")
```

3. `len()`: Count how many items are inside

`len()` returns the number of items in the list.

```
backpack = ["water", "rope", "flashlight"]
```

```
print(len(backpack))          # 3
print(f"You packed {len(backpack)} items.")
```

Use `len()` when:

- You want to show how many things the player packed
- You want to check if the list is empty
- You want to limit how big the backpack can be

Example:

```
if len(backpack) > 5:
    print("Your backpack is getting heavy.")
```

Common mistakes to avoid

- Forgetting parentheses: `len(backpack)` not `len backpack`
- Typos in remove: `"rope "` is not the same as `"rope"`
- Trying to remove something that is not there
- Capitalization differences: `"Water"` is not `"water"`

Map Help

String Based Map Formatting Guide

Most of your maps will be built by printing strings line by line. That is totally valid and recommended.

1. Think of each row as a string

Example 3 × 3 map:

```
row1 = ". . 1"  
row2 = ". 0 ."  
row3 = "X . ."
```

Then print line by line:

```
print(row1)  
print(row2)  
print(row3)
```

This is basically your “picture” of the world.

2. Use clear spacing

Good:

```
". . X . ."
```

Confusing:

```
". .X.."
```

Use spaces so you can see each tile.

. = empty wasteland

0 = you

X = safe zone

1, 2, 3 = special locations

3. Make a 5 × 5 map

Example:

```
row1 = ". . 1 . ."  
row2 = ". . . . ."  
row3 = "0 . . . ."  
row4 = ". . 2 . ."  
row5 = "X . . . 3"
```

Then:

```
print(row1)
print(row2)
print(row3)
print(row4)
print(row5)
```

4. Add a map key

After printing the map, explain what everything means.

```
print("Map Key:")
print("O = You (starting point)")
print("X = Final safe zone")
print("1 = Radio Tower")
print("2 = Old Market")
print("3 = Water Caves")
print(". = Empty wasteland")
```

You can use f-strings if you named your locations with variables.

5. Optional: simple border

You can add a border using extra prints.

```
print(" +-----+")
print(" " + row1)
print(" " + row2)
print(" " + row3)
print(" " + row4)
print(" " + row5)
print(" +-----+")
```

Keep it simple. The goal is for the map to be readable, not perfect.

input() Help

Quick Guide: `input()` in Python

`input()` lets your program ask the user a question and use their answer.

1. Basic input

```
name = input("What is your name? ")

print(f"Hello, {name}!")
```

What happens:

- Python shows the message
- The user types
- The answer is stored as a string

All input is text (a string), even if it looks like a number.

2. Converting input into numbers

If you want a number, you must convert the string.

```
age_text = input("How old are you? ")
age = int(age_text)

print(f"Next year, you will be {age + 1}.")
```

Or in one step:

```
age = int(input("How old are you? "))
```

Use `int()` for whole numbers, `float()` for decimals.

3. Cleaning up answers with `.lower()`

Users might type `YES`, `Yes`, `yEs`, and so on.

`.lower()` makes everything lowercase so it is easier to compare.

```
answer = input("Do you want to add another item? (yes/no): ")
answer = answer.lower()

if answer == "yes":
    print("Adding another item...")
else:
    print("Stopping here.")
```

4. Common mistakes to avoid

- Forgetting to convert to `int` or `float` before doing math
- Comparing `"Yes"` when you converted to lowercase
- Typos in prompts or variable names
- Putting spaces where you do not expect them

If something breaks, add simple prints to see what the input actually is.

LEGENDARY

U1L6 Mini Project – LEGENDARY MODE

“The Full Wasteland Expedition”

You’ve survived the basics. You’ve ventured beyond the safety of your settlement. Now, you’re ready for the **full wasteland expedition**, complete with an interactive backpack, a large-scale map, and rich lore behind every location.

Legendary Mode challenges you to use everything you’ve learned so far — and introduces a few new tools that prepare you for real game development later on.

This path is meant for students who want:

- More independence
- More challenge
- More creativity
- More code that *feels like an actual program*

You will build an evolving backpack using a **while loop**, create customizable locations, and print a full **10×10 map** of your world.

But remember:

- You can still use string-based maps if that feels right.
- Multi-dimensional maps (2D lists) are *optional*, but highly encouraged.

You can use the attached resources as well as any of your previous notes and materials to help you.

Legendary Resources:

- [append\(\), remove\(\), and len\(\)](#)
- [Map Help \(Legendary\)](#)
- [while Loops](#)
- [Advanced Backpack Features](#)

PART 1 – Interactive Backpack System

You’ll build a backpack that grows based on user choices.

Your tasks

1. Start with an empty list called **backpack**.
2. Use a **while loop** to repeatedly ask:
 - “What do you want to add to your backpack?”
 - “Add another item? (yes/no)”
3. Add each item using **.append()**.

4. Use `.lower()` so your loop accepts "YES, Yes, yes, yEs," etc.
5. Stop the loop when the user types "no".
6. Print:
 - total items (`len(backpack)`)
 - first item (`backpack[0]`)
 - last item (`backpack[-1]`)
7. Allow the user to remove one item if they want.

PART 2 – Creating Your 10×10 Wasteland Map

In Legendary mode, your map gets big — **10 rows, each 10 characters wide**.

You choose:

- your starting location (0)
- your final safe zone (X)
- **three major points of interest** (1, 2, 3)
- optional extra markers (#, *, additional numbers, terrain types)

You may build your map using:

Option A (Basic): String Rows

Ten strings → print row by row.

Option B (Advanced): 2D List (Grid)

A list of lists → print using a loop.

The 2D list method is fully *optional* but strongly encouraged for students who want to feel like they're building a real game map.

Option A – String-Based 10×10 Map (Basic)

Example:

```
row1 = ". . . 1 . . . . . ."
row2 = ". . . . . . . . . ."
row3 = ". 0 . . . . . . . ."
row4 = ". . . . . . . . . ."
row5 = ". . . 2 . . . . . ."
row6 = ". . . . . . . . . ."
row7 = ". . . . . . 3 . . . ."
row8 = ". . . . . . . . . ."
row9 = "X . . . . . . . . ."
row10 = ". . . . . . . . . ."
```

Then:

```
print(row1)
print(row2)
...
print(row10)
```

Option B – 2D List Map Generator (Advanced)

Here is the optional helper code students may use:

```
# -----
# MAP GENERATOR HELPERS
# -----

def make_empty_grid():
    return [["." for _ in range(10)] for _ in range(10)]

def print_map(grid):
    for row in grid:
        print(" ".join(row))

map_grid = make_empty_grid()

# Students decide these coordinates:
# map_grid[row][col] = "0"
# map_grid[row][col] = "X"
# map_grid[row][col] = "1"
# map_grid[row][col] = "2"
# map_grid[row][col] = "3"
```

This makes maps much easier to edit.

Students then add:

```
print_map(map_grid)
```

PART 3 – Location Names + Lore

You will name each important location and describe its lore.

Ask the user for:

- **Location 1 name + lore**
- **Location 2 name + lore**
- **Location 3 name + lore**

- **Safe zone name + lore**

Example structure:

```
loc1_name = input("Name location 1: ")  
loc1_lore = input("Describe location 1: ")
```

At the end, print a full **map key** using f-strings.

PART 4 – Final Story (5–8 Sentences)

Your story should feel like the beginning of a real adventure.

✓ Your story must:

- mention the safe zone by name
- include at least one backpack item using indexing
- reference a numbered location
- describe something about the wasteland
- include how many items you packed
- tie together your map + backpack + lore

✓ Optional extensions:

- reveal a secret at one of the locations
- introduce an NPC (non-player character)
- include a danger or challenge
- describe a path marked by * on your map
- include a twist (the safe zone is not what it seems)

Tone is completely up to you — funny, dark, hopeful, chaotic, sci-fi, or cinematic.

Level-Up Beyond Legendary (Optional Bonus)

If you want to go even further:

- Draw a path on the map using *
- Create weather effects (variables)
- Sort the items in your backpack
- Track health or stamina
- Add random events
- Implement a basic travel loop
- Add ASCII art splash screens
- Use multiple maps (cave, canyon, city, etc.)

None of these are required for full credit — they're just extra fun.

Starter code on the next page

Legendary Starter Code

Python

```
# =====
# U1L6 – LEGENDARY MODE STARTER FILE
# "THE FULL WASTELAND EXPEDITION"
# NOTE: This file includes COMMENTS ONLY.
# You will write ALL the code yourself.
# Use the Resource Pages provided to guide you:
#   • While Loop Quick Guide
#   • append/remove/len mini guide
#   • Map Formatting + 10x10 Map Examples
#   • F-String Reference Page
#   • Level Up Anytime Sheet
# =====

# -----
# PART 1 – INTERACTIVE BACKPACK SYSTEM
# -----

# Create an empty list for the backpack.
# (example: backpack = [])

# Write a WHILE LOOP that:
#   • asks the user for an item to add
#   • appends the item to the list using .append()
#   • asks "Add another item? (yes/no)"
#   • converts the response to lowercase using .lower()
#   • stops the loop when they type "no"

# After the loop:
#   • print how many total items (using len())
#   • print the FIRST item using index 0
#   • print the LAST item using index -1
#   • OPTIONAL: ask the user if they want to remove an item
#     (use .remove() if so)

# REMINDER:
# Use the While Loop Guide if you're stuck.
```

```

# Use the List Methods Mini Guide for append(), remove(), len().
# -----

# -----

# PART 2 – YOUR 10x10 WASTELAND MAP
# -----

# Choose ONE of the two map styles:

# OPTION A – String-Based Map:
#   Create 10 separate variables (row1, row2, etc.)
#   Each row is a STRING of 10 symbols separated by spaces.
#   You may use:
#       .   (empty ground)
#       0   (your starting point)
#       X   (your final safe zone)
#       1, 2, 3 (locations with lore)
#       *   (optional path marker)
#   When finished, print all 10 rows in order.

# OPTION B – 2D LIST MAP (recommended, but optional):
#   Create a 10x10 grid using a list of lists.
#   Each position should start as ".".
#   Replace grid positions with:
#       "0"  starting position
#       "X"  safe zone
#       "1"  first lore location
#       "2"  second lore location
#       "3"  third lore location
#   Write a function OR loop to print the map row-by-row
#   using " ".join(row) for neat formatting.

# IMPORTANT:
#   Do NOT type all the spaces manually unless you choose to.
#   Use the Map Formatting Resource for tips and shortcuts.

# -----

# -----

# PART 3 – LOCATION LORE + NAMES
# -----

# Ask the user to NAME each location:
#   • Location 1 name
#   • Description / lore for Location 1
#   • Location 2 name

```

```

#   • Description / lore for Location 2
#   • Location 3 name
#   • Description / lore for Location 3
#   • Safe Zone final name
#   • Safe Zone lore

# Store these in variables so you can use them later.
# Consider using f-strings to format your map key.

# -----

# -----
# PART 4 – FINAL STORY (5–8 SENTENCES)
# -----

# Write a story at the bottom of your program using print().
# Your story MUST include:
#   • The name of the safe zone
#   • At least one backpack item using INDEXING
#       (example pattern: backpack[0] or backpack[-1])
#   • A reference to at least ONE numbered location
#   • How many total items you packed (use len())
#   • A short description of the wasteland
#   • Something connecting the map + backpack + lore together

# OPTIONAL:
#   • Reveal a secret about one of the locations
#   • Add a plot twist
#   • Reference a path made of "*" on the map
#   • Use formatting / spacing tricks from the F-String Guide
# -----

# -----
# EXTRA OPTIONAL CHALLENGES (NOT REQUIRED)
# -----

# Add NPC characters.
# Add a health or stamina variable.
# Create weather effects (e.g., "acid rain" reduces visibility).
# Add random events using random.choice().
# Draw ASCII art splash screens.
# Create multiple maps (caves, ruins, etc.).
# -----

# End of Legendary Starter File

```

```
# Good luck - build something epic!
```

```
# =====
```

append(), remove(), and len()

Mini Guide: `append()`, `remove()`, and `len()`

These three list tools will show up everywhere in your Python projects.

1. `append()`: Add an item to the end

`append()` sticks a new item onto the end of your list.

```
backpack = ["water", "rope", "flashlight"]
```

```
backpack.append("first aid kit")
```

```
print(backpack)
```

```
# ["water", "rope", "flashlight", "first aid kit"]
```

Use `append()` when:

- You want to grow your list one item at a time
- The user gives you a new item with `input()`
- You are collecting options, choices, or inventory

2. `remove()`: Take out a specific item

`remove()` deletes the first match of that value from the list.

```
backpack = ["water", "rope", "flashlight", "rope"]
```

```
backpack.remove("rope")
```

```
print(backpack)
```

```
# ["water", "flashlight", "rope"]
```

Notes:

- It looks for the value, not the index
- It only removes the first match
- If the item is not in the list, you will get an error

Safer version:

```
item_to_remove = "rope"
```

```
if item_to_remove in backpack:
```

```
    backpack.remove(item_to_remove)
```

```
else:
```

```
    print(f"{item_to_remove} is not in your backpack.")
```

3. `len()`: Count how many items are inside

`len()` returns the number of items in the list.

```
backpack = ["water", "rope", "flashlight"]
```

```
print(len(backpack))          # 3
print(f"You packed {len(backpack)} items.")
```

Use `len()` when:

- You want to show how many things the player packed
- You want to check if the list is empty
- You want to limit how big the backpack can be

Example:

```
if len(backpack) > 5:
    print("Your backpack is getting heavy.")
```

Common mistakes to avoid

- Forgetting parentheses: `len(backpack)` not `len backpack`
- Typos in remove: `"rope "` is not the same as `"rope"`
- Trying to remove something that is not there
- Capitalization differences: `"Water"` is not `"water"`

Map Help (Legendary)

Advanced Map Help – String Maps and 2D List Maps

In Legendary mode, your map grows to 10 by 10. There are two main ways to build it:

- **String based map** – easiest to start with
- **2D list based map** – more advanced, great for grid games

You can earn full credit using either method. The 2D list method is just a great power-up.

1. String Based 10 × 10 Map

Here, each row is just a string.

```
row1 = ". . . 1 . . . . ."  
row2 = ". . . . . . . . ."  
row3 = ". 0 . . . . . . ."  
row4 = ". . . . . . . . ."  
row5 = ". . . 2 . . . . ."  
row6 = ". . . . . . . . ."  
row7 = ". . . . . 3 . . ."  
row8 = ". . . . . . . . ."  
row9 = "X . . . . . . . ."  
row10 = ". . . . . . . . ."
```

Print it like this:

```
print(row1)  
print(row2)  
print(row3)  
print(row4)  
print(row5)  
print(row6)  
print(row7)  
print(row8)  
print(row9)  
print(row10)
```

This works well if you plan your positions ahead of time.

2. 2D List Map – A Grid in Code

A **2D list** is a list of rows, where each row is a list of cells.

You can think of it like a table:

```
grid = [
    [".", ".", ".", ".", ".", ".", ".", ".", ".", "."],
    [".", ".", ".", ".", ".", ".", ".", ".", ".", "."],
    [".", ".", ".", ".", ".", ".", ".", ".", ".", "."],
    [".", ".", ".", ".", ".", ".", ".", ".", ".", "."],
    [".", ".", ".", ".", ".", ".", ".", ".", ".", "."],
    [".", ".", ".", ".", ".", ".", ".", ".", ".", "."],
    [".", ".", ".", ".", ".", ".", ".", ".", ".", "."],
    [".", ".", ".", ".", ".", ".", ".", ".", ".", "."],
    [".", ".", ".", ".", ".", ".", ".", ".", ".", "."],
    [".", ".", ".", ".", ".", ".", ".", ".", ".", "."]
]
```

Each cell is `grid[row][column]`:

- `grid[0][0]` is the top left
- `grid[2][3]` is row 3, column 4
- `grid[9][9]` is the bottom right

3. Placing symbols in the 2D list

You can change individual cells like this:

```
grid[2][2] = "0" # starting point
grid[8][0] = "X" # safe zone
grid[0][3] = "1" # location 1
grid[4][5] = "2" # location 2
grid[6][7] = "3" # location 3
```

You get full control over exact positions.

4. Printing the 2D list map

Use a loop and `join` to print:

```
for row in grid:
    print(" ".join(row))
```

This turns, for example, `[ ".", "0", "." ]` into `". 0 ."`.

5. Creating an empty grid with code

Instead of typing all rows by hand, you can generate the grid:

```
grid = [["." for _ in range(10)] for _ in range(10)]
```

This creates a 10 by 10 grid filled with ".".

You can then place symbols:

```
grid[2][2] = "O"  
grid[8][0] = "X"  
grid[0][3] = "1"
```

Then print with the loop shown above.

6. Optional: Add a path

You can mark a path using "*":

```
grid[2][2] = "O"  
grid[3][2] = "*"  
grid[4][2] = "*"  
grid[5][2] = "*"  
grid[6][2] = "*"  
grid[7][2] = "*"  
grid[8][0] = "X"
```

This can represent your character's trail across the wasteland.

7. When to use each method

String rows

- Easier to understand
- Good for simple maps
- You control each row manually

2D list

- Better for bigger maps
- Easier to change single cells
- Great for future grid games

You can choose either method for this project. 2D lists are encouraged if you feel ready.

while Loops

While Loop Explainer – “Keep Going Until...”

A **while loop** lets your code repeat something again and again **until a condition changes**.

Think of it like this:

While I am hungry, keep eating.

While the game is running, keep moving.

While the player says “yes,” keep asking questions.

1. Basic pattern

```
while condition_is_true:
    # do something
    # maybe update the condition
```

If the condition never becomes False, the loop keeps running forever. That is called an **infinite loop**.

2. Simple example

```
count = 0

while count < 3:
    print("Loop is running...")
    count = count + 1
```

What this does:

- Starts count at 0
- Runs while count is less than 3
- After three runs, count is 3, the condition is no longer true, and the loop stops

3. While True with break

Sometimes we want a loop that keeps going until we decide to stop **inside** the loop.

That is where **while True** and **break** pair together.

```
while True:
    response = input("Type 'quit' to stop: ")

    if response == "quit":
        print("Stopping the loop.")
        break
```

```
print("You typed:", response)
```

- `while True` means “always run this”
- `break` is the escape hatch that stops the loop

4. Backpack style loop (pattern only)

Here is a pattern similar to what you will use in Legendary mode:

```
backpack = []
```

```
while True:
    # ask for an item
    # add it to the backpack with append

    # ask if they want to add another item
    # convert the answer to lowercase with .lower()

    # if they say "no", use break to stop the loop
```

You will fill this in using the While Loop and List Methods guides.

5. Common mistakes to watch for

- Forgetting to update the variable inside the loop
- Forgetting `break`, so it loops forever
- Writing a condition that is never true or never false
- Indentation errors

If your program never stops or feels “stuck,” check your loop condition and where you use `break`.

Advanced Backpack Features

Advanced Backpack Features – Sorting and Searching

If your backpack is full of items, you might want to sort them or search through them. These are **optional advanced skills** that can make your project feel more like a real game.

1. Sorting your backpack

You can sort the list alphabetically.

```
backpack = ["rope", "water", "flashlight", "bandages"]
```

```
backpack.sort()
```

```
print(backpack)
# ['bandages', 'flashlight', 'rope', 'water']
```

Notes:

- `sort()` changes the original list in place
- It sorts alphabetically (A to Z) by default

If you do not want to change the original list, you can use `sorted()` instead:

```
sorted_backpack = sorted(backpack)
print(sorted_backpack)
```

2. Checking if an item is in the backpack

Use the `in` keyword:

```
if "water" in backpack:
    print("You have water.")
else:
    print("You might get thirsty.")
```

This is helpful before you use `remove()`.

3. Finding where an item is (index)

If you want to know where an item is in the list, use `.index()`:

```
position = backpack.index("flashlight")
print(position)
```

Remember:

- Index starts at 0
- If the item is not in the list, `.index()` will cause an error
- It only returns the first match

Safer version:

```
item = "flashlight"

if item in backpack:
    position = backpack.index(item)
    print(f"{item} is at position {position}.")
else:
    print(f"{item} is not in your backpack.")
```

4. Searching with a loop

You can also search by looping through the list:

```
search_for = "map"
found = False

for item in backpack:
    if item == search_for:
        found = True
        break

if found:
    print(f"{search_for} is in your backpack.")
else:
    print(f"{search_for} is not in your backpack.")
```

This is useful if you want to do something more complex during the search.

5. Ideas for using advanced backpack features in your story

- Sort items before you leave camp, so everything is organized
- Check if you have an item before entering a dangerous area
- Print a warning if a key item is missing
- Use searching to decide different story branches

Example idea:

```
if "flashlight" in backpack:  
    print("You switch on your flashlight and enter the dark tunnel.")  
else:  
    print("Without a flashlight, the tunnel is too risky to enter.")
```

Peer Review

Peer Review: Wasteland Adventure Project

Before we begin our peer review session, you will complete a short [Skill Match Survey](#). This survey helps us understand:

- What level you chose for the project
- What skills you feel confident helping others with
- What skills you want help with

Based on your answers, you will be paired with a classmate who can support you in the areas you need — and who you can also support in return. The goal is not just to get feedback, but to **share strengths**, learn from each other, and level up together.

Once paired, you and your partner will complete the **Peer Feedback Form**, where you will:

- Celebrate what's going well
- Check each other's project requirements
- Offer supportive, actionable feedback
- Talk about challenges and areas to revise
- Write a short log of what help you offered or received
- Optional: leave helpful comments directly in code (ex: labeling where a loop was fixed, explaining a map formatting tip, etc.)

At the end of the peer review, you'll reflect on:

- What your partner helped you understand
- Whether their support helped you move past an error or confusion
- How confident you now feel with the project skills
- What personal collaboration skills helped you today (for example:
 - asking clear questions
 - initiating conversation
 - showing patience
 - trying something new just to test it
 - taking a "brave moment" to ask for help
 - explaining your thinking out loud
 - offering encouragement)

Peer review is a critical part of this project because **coders learn best by helping each other**. This is how real programming teams work — sharing ideas, testing solutions, giving feedback, and building stronger projects together.

So be kind, be curious, be brave, and support your partner.

Together, you can create something stronger than either of you could alone.